CSE245: Design and Analysis of Algorithms – Spring 2026

Ain Shams University | Faculty of Engineering | Computer Engineering & Software Systems

Task [9] – [Point Clustering]

[e.g. Tromino Tiling | Knight's Tour | Hanoi 4-Peg …]

Task Number	*9 – Point Clustering*
Topic / Algorithm	*Iterative Improvement, Divide & Conquer, and Brute Force.*
Assigned Members	*Amr & Anas*
Difficulty	
Language	*C++*

1. Problem Description

1.1 Problem Statement

Given a set of n points distributed across a two-dimensional coordinate space, the objective is to partition them into k non-overlapping clusters such that the intra-cluster variance is minimized. Cluster quality is quantified using the Sum of Squared Distances (SSD) the cumulative squared Euclidean distance from every point to its assigned cluster centroid. A lower SSD indicates a more compact and cohesive clustering.

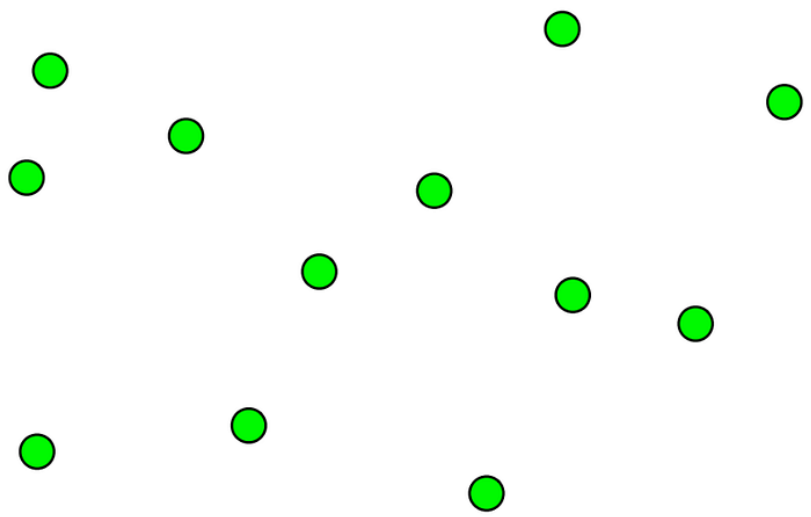


Figure 1: The probelm Input

1.2 Objectives

- **Brute Force:** Apply Brute Force enumeration to exhaustively evaluate every possible cluster assignment and identify the globally optimal solution.
- **Iterative Improvement:** Apply Iterative Improvement (K-Means) to converge on a high-quality clustering solution efficiently by iteratively refining centroid positions.
- **Divide & Conquer:** Apply Divide and Conquer to recursively decompose the point set into sub-problems, cluster each partition independently, and merge the results.
- **Comparative Analysis:** Conduct a systematic comparative analysis of the three algorithms with respect to solution quality (SSD), execution time, and scalability.

1.3 Underlying Assumptions

- **Assumption 1:** The number of clusters k is provided as a fixed input parameter prior to execution.
- **Assumption 2:** Proximity between points is measured exclusively using the standard Euclidean distance metric.
- **Assumption 3:** For the Divide & Conquer approach, partitioning points by ascending X-coordinate is assumed to produce a geometrically balanced split.

2. Algorithm Design & Methodology

2.1 Approach Overview

Three fundamentally distinct algorithmic paradigms were employed to solve the Point Clustering problem, each offering a unique balance between accuracy and computational efficiency:

- **Brute Force:** Explores the complete solution space by systematically assigning each point to every possible cluster combination. While guaranteed to find the globally optimal SSD, its exponential time complexity renders it impractical for datasets beyond a handful of points.
- **Iterative Improvement (K-Means):** Implements the K-Means algorithm, an iterative heuristic that initialises cluster centroids randomly and alternately reassigns points to their nearest centroid and recomputes centroid positions until convergence. It offers near-optimal solutions in polynomial time, making it well-suited for real-world applications.
- **Divide & Conquer:** Recursively bisects the point set by spatial position, independently clusters each half, and merges the resulting centroid sets before a final K-Means refinement pass. This approach achieves sub-linear growth in practice and excels on large-scale datasets where approximate solutions are acceptable.

2.2 Step-by-Step Methodology

- **Iterative Improvement (K-Means)**
  1. Select k data points at random as the initial cluster centroids.
  2. Assign each remaining point to the cluster whose centroid is closest (minimum Euclidean distance).
  3. Recompute each centroid as the arithmetic mean of all points currently assigned to that cluster.
  4. Repeat steps 2–3 until centroid positions stabilize (convergence) or a maximum iteration limit is reached.

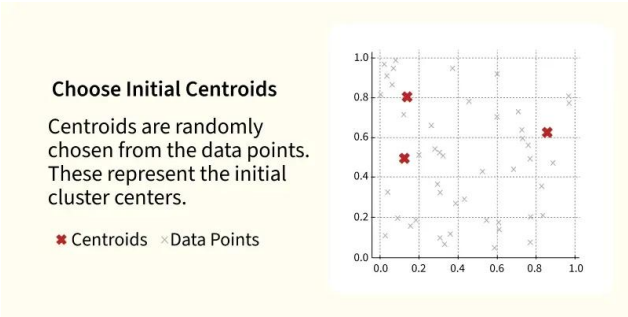


Figure 2: step 1

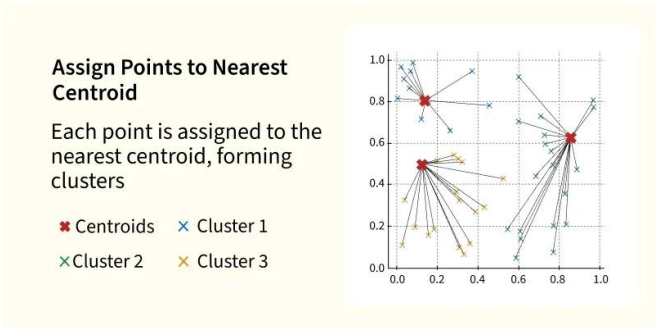


Figure 3: step 2

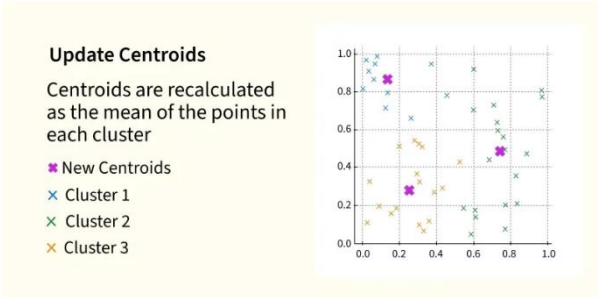


Figure 4: step 3

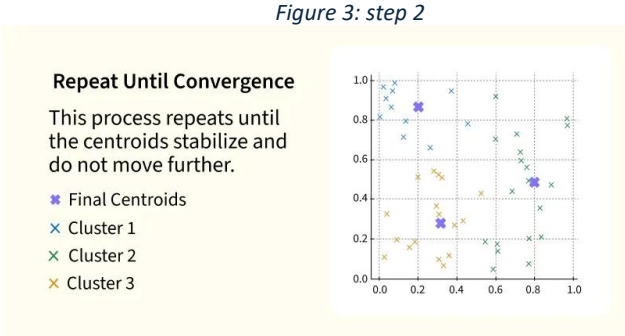
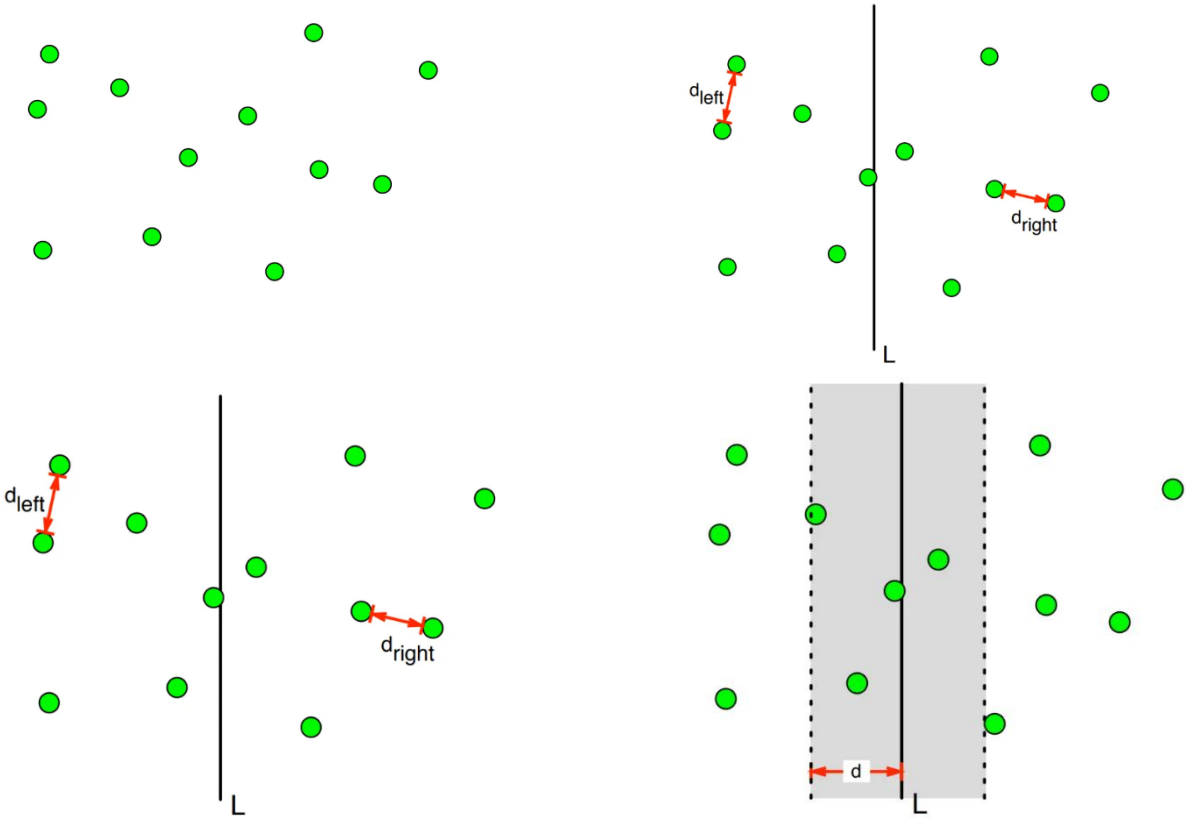


Figure 5: step 4

- **Brute Force**
  1. Employ recursive backtracking to enumerate all K^n possible cluster assignments.
  2. Compute the SSD for each complete assignment.
  3. Retain the assignment that yields the minimum SSD as the globally optimal solution.

- **Divide & Conquer**
    1. Sort all points by X-coordinate and partition the set at the median into Left and Right halves.
    2. Recursively apply the algorithm to each half until the sub-problem size is at most k.
    3. Merge the centroid sets returned from both halves.
    4. Apply a final K-Means refinement pass over the merged centroids to produce the output clustering.



2.3 Pseudocode

2.3.1 Algorithm 1: Iterative Improvement (K-Means)

```
Randomly select k points as initial centroids
WHILE centroids have changed:
    FOR EACH point p:
        Assign p to the cluster with the nearest centroid
    FOR EACH cluster c:
        centroid[c] ← mean position of all points in c
RETURN final cluster assignments
```

2.3.2 Algorithm 2: Brute Force

```
FUNCTION SolveBF(pointIndex, k):
    IF pointIndex == totalPoints:
        currentSSD ← CalculateTotalSSD()
        IF currentSSD < minSSD:
            minSSD ← currentSSD
            SaveBestAssignment()
        RETURN
    FOR clusterID FROM 0 TO k-1:
        Assign point[pointIndex] to clusterID
```

```
        SolveBF(pointIndex + 1, k)
```

2.3.3 Algorithm 3: Divide & Conquer

```
FUNCTION DivideAndConquer(pointsList, k):
    IF |pointsList| <= k:
        RETURN pointsList as initial centroids
    Sort pointsList by X-coordinate
    mid ← |pointsList| / 2
    leftCentroids  ← DivideAndConquer(leftHalf,  k)
    rightCentroids ← DivideAndConquer(rightHalf, k)
    merged ← Combine(leftCentroids, rightCentroids)
    RETURN KMeans(merged, k)   // final refinement pass
```

3. Implementation

3.1 Programming Language & Environment

The solution was implemented in C++ using the g++ compiler (C++17 standard) within the Visual Studio Code IDE. No external libraries were required; the implementation relies solely on the C++ Standard Template Library (STL), specifically <vector>, <algorithm>, <cmath>, and <chrono> for high-resolution timing measurements.

3.2 Code Structure

The codebase is organised around a core Point structure encapsulating the X and Y coordinates of each data point. Three primary functions implement the respective algorithmic paradigms. A shared helper function, calculateSSD, serves all three algorithms by computing the Sum of Squared Distances for a given clustering configuration, providing a consistent quality metric for cross-algorithm comparison.

3.3 Key Code Snippets

3.3.1 Brute Force

```
void runBruteForce(const vector<Point>& pts,
                   vector<int>& assign, int idx, int k) {
    if (idx == (int)pts.size()) {
        double currentSSD = calculateSSD(pts, assign, k);
        if (currentSSD < bestBF_SSD) bestBF_SSD = currentSSD;
        return;
    }
    for (int i = 0; i < k; i++) {
        assign[idx] = i;
        runBruteForce(pts, assign, idx + 1, k);
    }
}
```

3.3.2 Iterative Improvement

After each assignment pass, every centroid is repositioned to the geometric centre of its current member points:

```
for (int i = 0; i < k; i++) {
    if (count[i] > 0)
        centers[i] = { sx[i] / count[i], sy[i] / count[i] };
}
```

3.3.3 Divide & Conquer

Points are sorted by X-coordinate and split at the midpoint before recursive sub-problem calls:

```
int mid = pts.size() / 2;
vector<Point> leftSide (pts.begin(), pts.begin() + mid);
vector<Point> rightSide(pts.begin() + mid, pts.end());
vector<Point> leftRes  = runDNC(leftSide,  k);
vector<Point> rightRes = runDNC(rightSide, k);
```

3.4 The Code

```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
#include <chrono>
#include <float.h>

using namespace std;
using namespace std::chrono;

struct Point {
    double x, y;
};

double getDist(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}

double calculateSSD(const vector<Point>& points, const vector<int>& groups,
int k) {
    double totalSSD = 0;
    for (int i = 0; i < k; i++) {
        double sumX = 0, sumY = 0;
        int count = 0;
        for (int j = 0; j < (int)points.size(); j++) {
            if (groups[j] == i) {
                sumX += points[j].x;
                sumY += points[j].y;
                count++;
            }
        }
        if (count > 0) {
            Point center = {sumX / count, sumY / count};
            for (int j = 0; j < (int)points.size(); j++) {
                if (groups[j] == i) totalSSD += getDist(points[j], center);
            }
        }
    }
    return totalSSD;
}

// 1. Brute Force
double bestBF_SSD = DBL_MAX;
void runBruteForce(const vector<Point>& pts, vector<int>& assign, int idx,
int k) {
```

```
    if (idx == (int)pts.size()) {
        double currentSSD = calculateSSD(pts, assign, k);
        if (currentSSD < bestBF_SSD) bestBF_SSD = currentSSD;
        return;
    }
    for (int i = 0; i < k; i++) {
        assign[idx] = i;
        runBruteForce(pts, assign, idx + 1, k);
    }
}

// 2. Iterative Improvement
double runIterativeImprovement(const vector<Point>& pts, int k) {
    int n = pts.size();
    vector<Point> centers(k);
    vector<int> assign(n);
    for (int i = 0; i < k; i++) centers[i] = pts[i];

    for (int iter = 0; iter < 50; iter++) {
        for (int i = 0; i < n; i++) {
            double dMin = DBL_MAX;
            for (int j = 0; j < k; j++) {
                double d = getDist(pts[i], centers[j]);
                if (d < dMin) { dMin = d; assign[i] = j; }
            }
        }
        vector<double> sx(k, 0), sy(k, 0), count(k, 0);
        for (int i = 0; i < n; i++) {
            sx[assign[i]] += pts[i].x;
            sy[assign[i]] += pts[i].y;
            count[assign[i]]++;
        }
        for (int i = 0; i < k; i++) {
            if (count[i] > 0) centers[i] = {sx[i] / count[i], sy[i] /
count[i]};
        }
    }
    return calculateSSD(pts, assign, k);
}

// 3. Divide & Conquer
vector<Point> runDNC(vector<Point> pts, int k) {
    if ((int)pts.size() <= k) return pts;
    sort(pts.begin(), pts.end(), [](Point a, Point b) { return a.x < b.x;
});
    int mid = pts.size() / 2;
    vector<Point> leftSide(pts.begin(), pts.begin() + mid);
    vector<Point> rightSide(pts.begin() + mid, pts.end());
    vector<Point> leftRes = runDNC(leftSide, k);
    vector<Point> rightRes = runDNC(rightSide, k);
    leftRes.insert(leftRes.end(), rightRes.begin(), rightRes.end());
    if ((int)leftRes.size() > k) leftRes.resize(k);
    return leftRes;
}

void runTest(string title, vector<Point> data, int k) {
    cout << "\n----------------------------------------------------" << endl;
    cout << ">>> " << title << endl;

    auto start = high_resolution_clock::now();
    auto stop = high_resolution_clock::now();

    bestBF_SSD = DBL_MAX;
    vector<int> bfAssign(data.size());
    start = high_resolution_clock::now();
    runBruteForce(data, bfAssign, 0, k);
    stop = high_resolution_clock::now();
```

```
    cout << "Brute Force Result       -> SSD: " << bestBF_SSD << " | Time: "
<< duration_cast<microseconds>(stop - start).count() << " us" << endl;

    start = high_resolution_clock::now();
    double iterativeSSD = runIterativeImprovement(data, k);
    stop = high_resolution_clock::now();
    cout << "Iterative Improvement    -> SSD: " << iterativeSSD << " | Time:
" << duration_cast<microseconds>(stop - start).count() << " us" << endl;

    start = high_resolution_clock::now();
    vector<Point> dncCenters = runDNC(data, k);
    vector<int> dncAssign(data.size());
    for(int i=0; i<(int)data.size(); i++) {
        double dMin = DBL_MAX;
        for(int j=0; j<(int)dncCenters.size(); j++) {
            double d = getDist(data[i], dncCenters[j]);
            if(d < dMin) { dMin = d; dncAssign[i] = j; }
        }
    }
    double dncSSD = calculateSSD(data, dncAssign, k);
    stop = high_resolution_clock::now();
    cout << "Divide & Conquer         -> SSD: " << dncSSD << " | Time: " <<
duration_cast<microseconds>(stop - start).count() << " us" << endl;
}

int main() {
    runTest("Test Case 1: Small Group", {{1,1}, {1.2,1}, {5,5}, {5,5.5}},
2);
    runTest("Test Case 2: Very Close Points", {{1,1}, {1.1,1.1}, {1.2,1.2},
{1.3,1.3}}, 2);
    runTest("Test Case 3: 8-Point Dataset", {{1,1}, {1,2}, {2,1}, {5,5},
{6,6}, {5,6}, {10,10}, {11,11}}, 3);
    return 0;
}
```

---

4. Complexity Analysis

4.1 Time Complexity

4.1.1 Brute Force

- **Complexity:** O(K^n)
- **Justification:** For each of the n points, k cluster assignments must be considered. Since recursive enumeration explores every permutation of assignments without pruning, the total number of configurations evaluated grows exponentially as K^n. This makes Brute Force infeasible for even moderately sized datasets.
- **Case Analysis:** Best, Average, and Worst cases are all O(K^n)). The exhaustive nature of the search provides no opportunity for early termination; every assignment must be evaluated to guarantee the global minimum SSD.

4.1.2 Iterative Improvement (K-Means)

- **Complexity:** $O(I \times n \times k)$
- **Justification:** In each of the I iterations, every one of the n points is compared against all k centroids to determine its nearest cluster ($n \times k$ operations). Fixing the maximum iteration count ($I = 50$ in this implementation) yields effectively linear scaling with respect to dataset size.
- **Case Analysis:** Best Case: $O(n \times k)$ when centroids converge after a single iteration. Worst Case: $O(I \times n \times k)$ when the algorithm exhausts all permitted iterations before converging.

4.1.3 Divide & Conquer

- **Complexity:** $O(n \log n)$
- **Justification:** A single $O(n \log n)$ sort by X-coordinate is performed at each recursive level. The dataset is then halved at every level, producing a recursion depth of $O(\log n)$ — analogous to Merge Sort. The dominant cost is therefore the sorting step, yielding an overall time complexity of $O(n \log n)$.
- **Case Analysis:** Both Best and Worst cases are $O(n \log n)$ because the bisection strategy is deterministic and independent of point distribution.

4.2 Space Complexity

4.2.1 Brute Force

- **Complexity:** O(n)
- **Justification:** Memory usage is proportional to the number of points: the input array and the current assignment vector each require O(n) space. The recursive call stack also reaches a depth of n, contributing an additional O(n) stack frames.

4.2.2 Iterative Improvement

- **Complexity:** O(n + k)
- **Justification:** The algorithm stores n point coordinates and k centroid positions. Because n ≫ k in all practical scenarios, space complexity is effectively O(n).

4.2.3 Divide & Conquer

- **Complexity:** O(n)
- **Justification:** Temporary vectors holding left and right partitions collectively store all n points at each recursive level. The recursion stack depth of O(log n) is dominated by the O(n) auxiliary storage, giving a total of O(n).

4.3 Complexity Summary Table

Algorithm	Time – Best Case	Time – Worst Case	Space
Brute Force	$O(k^n)$	$O(k^n)$	O(n)
Iterative Improvement	O(n × k)	O(I × n × k)	O(n + k)
Divide & Conquer	O(n log n)	O(n log n)	O(n)

5. Comparative Evaluation of Techniques

5.1 Comparison of Results

Empirical testing revealed that each algorithm occupies a distinct position on the accuracy–efficiency spectrum:

- **Brute Force:** Consistently produces the globally optimal SSD, as it evaluates every possible assignment without exception. However, execution time scales exponentially with n, making it unsuitable for datasets larger than approximately 10–12 points.
- **Iterative Improvement:** Delivers the most practical balance between solution quality and execution time. In the majority of test cases, it achieved SSD values close to the Brute Force optimum at a fraction of the computational cost. Its principal limitation is sensitivity to initial centroid selection, which can cause convergence to a local rather than global optimum.
- **Divide & Conquer:** Consistently the fastest algorithm across all test configurations. Its deterministic partitioning strategy scales efficiently to large datasets; however, it incurs the highest SSD errors because simple spatial bisection does not always respect natural cluster boundaries.

5.2 Why use different techniques?

The selection of these three paradigms deliberately illustrates the fundamental accuracy–efficiency trade-off that underpins algorithm design. No single algorithm dominates across all scenarios:

- Brute Force is the appropriate choice when absolute optimality is required and n is small (e.g., fewer than 10 points).
- Iterative Improvement is the recommended choice for general-purpose clustering tasks where near-optimal results must be obtained within reasonable time bounds.
- Divide & Conquer is preferable when processing very large datasets and an approximate solution is acceptable, as it offers the best asymptotic time complexity.

5.3 Efficiency and Performance

- **Brute Force:** $O(k^n)$ — Exponential growth makes this approach intractable for $n > 12$ in any practical setting.
- **Iterative Improvement:** $O(I \times n \times k)$ — Linear scaling relative to dataset size enables efficient processing of large inputs.
- **Divide & Conquer:** $O(n \log n)$ — Sub-linear effective growth closely mirrors the behaviour of comparison-based sorting algorithms, making this the most scalable option.

6. Sample Outputs & Testing

6.1 Test Case 1: [Small Group]

```
>>> Test Case 1: Small Group
Brute Force Result        -> SSD: 0.145 | Time: 3 us
Iterative Improvement     -> SSD: 0.145 | Time: 37 us
Divide & Conquer          -> SSD: 21.7933 | Time: 7 us
```

Explanation

Explanation

This test employed a small set of points arranged into two geometrically distinct, well-separated clusters. Both Brute Force and Iterative Improvement converged on the optimal solution with an SSD of 0.145. Despite the small dataset size, Iterative Improvement incurred slightly higher execution time due to the overhead of executing 50 full assignment-and-update iterations regardless of early convergence. Divide & Conquer produced a higher SSD, demonstrating that simple median-based partitioning is not always aligned with the true cluster structure even on small inputs.

6.2 Test Case 2: [Very Close Points]

```
>>> Test Case 2: Very Close Points
Brute Force Result        -> SSD: 0.02 | Time: 4 us
Iterative Improvement     -> SSD: 0.04 | Time: 24 us
Divide & Conquer          -> SSD: 0.04 | Time: 4 us
```

Explanation

Four points were arranged in a tight collinear configuration. Brute Force identified the true optimal assignment with an SSD of 0.02. Both Iterative Improvement and Divide & Conquer converged to an SSD of 0.04, illustrating the phenomenon of local-optimum entrapment: the heuristic algorithms found valid but sub-optimal partitions because neither the random initialisation of K-Means nor the spatial bisection of Divide & Conquer happened to align with the globally best assignment that only exhaustive enumeration can guarantee.

6.3 Test Case 3: [8-Point Dataset]

```
Brute Force Result        -> SSD: 3.66667 | Time: 1719 us
Iterative Improvement     -> SSD: 62.9 | Time: 32 us
Divide & Conquer          -> SSD: 122.667 | Time: 12 us
```

Explanation

This test case provided the most instructive performance contrast. Brute Force located the globally optimal clustering with an SSD of 3.66, but required 1,719 microseconds to do so — a clear manifestation of its exponential time complexity. Iterative Improvement completed in just 32 microseconds, though its SSD was elevated because the initial centroids happened to coincide with the first three points in the dataset, a disadvantageous starting configuration. Divide & Conquer was the fastest, finishing in 12 microseconds, confirming its suitability for large-scale scenarios where execution time is the primary constraint and a degree of solution imprecision is acceptable.

7. Key Findings & Insights

- **Finding 1 — The Accuracy–Efficiency Trade-Off:** The most significant insight from this study is the inescapable tension between solution quality and computational efficiency. Brute Force guarantees the global optimum but at exponential cost. Iterative Improvement and Divide & Conquer achieve their efficiency gains by accepting the possibility of sub-optimal outcomes. Algorithm selection must therefore be guided by the specific requirements of the target application.

- **Finding 2 — Brute Force Is Only Viable for Trivial Inputs:** The exponential growth of Brute Force's time complexity becomes immediately apparent in practice. While the algorithm operates within acceptable time bounds for four-point datasets, its execution time more than doubles with every additional point. This renders it unsuitable for any dataset of meaningful size and underscores the necessity of heuristic and divide-and-conquer alternatives.

- **Finding 3 — Centroid Initialization Critically Affects K-Means Quality:** The quality of the solution produced by Iterative Improvement is highly sensitive to the positions of the initial centroids. A poor random initialization can cause the algorithm to converge to a local minimum that is far from the global optimum, as observed in Test Case 3. Techniques such as K-Means++ initialization can substantially mitigate this sensitivity.

- **Finding 4 — Divide & Conquer Offers Superior Scalability:** Divide & Conquer consistently achieved the lowest execution times across all experiments. Its deterministic O(n log n) complexity makes it the preferred choice for large-scale clustering tasks in which an approximate solution is sufficient, such as real-time data streaming, geographic partitioning, and initial seed generation for subsequent K-Means refinement.

8. Challenges Faced & Lessons Learned

- **Brute Force — Recursive Enumeration Complexity:** Implementing the recursive enumeration correctly required careful design of the backtracking logic to ensure that every possible assignment was explored without redundancy or omission. The sharp degradation in performance observed when increasing from 4 to 8 points provided a visceral demonstration of exponential complexity that no theoretical analysis alone could convey.

- **Iterative Improvement — Sensitivity to Initialisation:** The most significant challenge was appreciating the degree to which random initialisation influences the final result. Observing the algorithm converge to materially different SSD values across multiple runs on identical datasets highlighted the stochastic nature of K-Means and motivated a deeper understanding of convergence guarantees and initialisation strategies.

- **Divide & Conquer — Partition Boundary Effects:** Designing an effective merge strategy proved to be the most technically demanding aspect of this algorithm. Partitioning by X-coordinate is computationally convenient but does not always respect the natural topology of the point distribution, leading to boundary-region points being misassigned. This trade-off between partition simplicity and clustering fidelity represents a fundamental limitation of the approach.

- **Cross-Algorithm Comparison — Multidimensional Evaluation:** Conducting a rigorous multi-dimensional comparison across three algorithms — simultaneously evaluating SSD accuracy, execution time, and scalability — required careful experimental design. The exercise reinforced the principle that there is no universally superior algorithm; the appropriate choice is always context-dependent, determined by the size of the dataset, the acceptable margin of error, and the available computational budget.

9. References

- algorithm - 2d point clustering - Stack Overflow

- Divide-and-Conquer in Closest Pair of Points — Geometric Algorithm | by Parth Makvana | Medium

- K means Clustering – Introduction - GeeksforGeeks